

MÓDULO 5

Funciones en Python

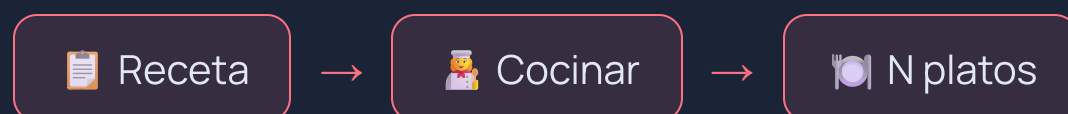
Sesión 5 · 3 horas — objetivo: definir y llamar funciones, usar parámetros/return, entender scope y documentar con docstrings.

¿Qué es una función?

Un bloque de código con nombre que realiza una tarea y puede ejecutarse cuantas veces se necesite.

Analogía — La receta de cocina

Defines la receta UNA vez y la usas cuantas veces quieras sin reescribirla. La función es igual: define → llama → reutiliza.



Reutilización

DRY — Don't Repeat Yourself



Organización

Divide el programa en partes manejables



Legibilidad

Más fácil de leer y entender



Pruebas

Cada parte se prueba independiente

Definir y llamar funciones

Primero defines (def), luego llamas. El código NO se ejecuta hasta que la llamas.

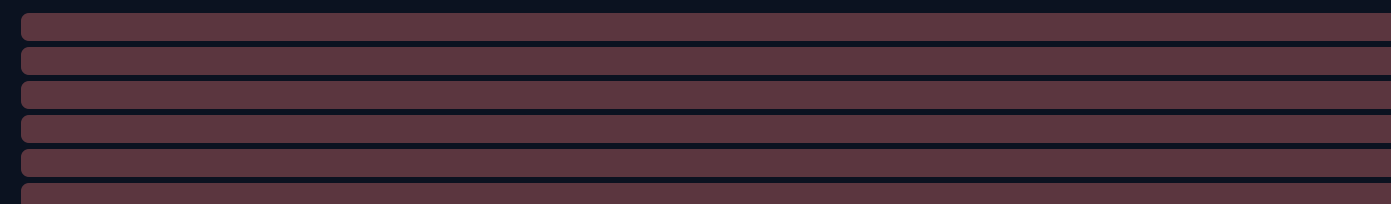
1. DEFINICIÓN (solo define, no ejecuta)

```
def mostrar_menu():  
    print("=" * 25)  
    print(" MENÚ DEL SISTEMA")  
    print("1. Ingresar datos")  
    print("2. Ver reporte")  
    print("=" * 25)
```

2. LLAMADA (aquí sí se ejecuta)

```
mostrar_menu()    # primera vez  
mostrar_menu()    # reutilización
```

Sin funciones



12 líneas repetidas



Con función



¡solo 2 líneas!

⚠ Define siempre la función ANTES de llamarla. Si la llamas antes, Python lanza NameError.

Parámetros, argumentos y valores por defecto

Parámetro = variable en el def · **Argumento** = valor real al llamar

```
#           parámetros
#           ↓   ↓
def saludar(nombre, cargo):
    print(f"Hola {nombre} ({cargo})")
#           argumentos
#           ↓       ↓
saludar("Hebert", "Instructor")
```

```
# Valor por defecto
def calcular_iva(precio, tasa=0.13):
    return precio * tasa
print(calcular_iva(100))      # 13.0
print(calcular_iva(100, 0.16)) # 16.0
```

```
# Argumentos por nombre – orden libre
def pedido(producto, cantidad, precio):
    print(f"{cantidad}x {producto} = Bs. {cantidad*precio:.2f}")

pedido("Cuaderno", 3, 15.0)      # posicional
pedido(precio=15.0, producto="Cuaderno",
       cantidad=3)              # por nombre
```

```
def f(a, b, c=10) ✓
```

```
def f(a, c=10, b) ✗
```

💡 Los parámetros con valor por defecto SIEMPRE van al final.

Valores de retorno – return

return permite que la función entregue un resultado para usar en el resto del programa.

❌ Con print – no reutilizable

```
def sumar(a, b):  
    print(a + b) # solo muestra  
r = sumar(3, 4) # imprime 7  
print(r * 2)    # None*2 ← ERROR
```

✅ Con return – reutilizable

```
def sumar(a, b):  
    return a + b # devuelve  
r = sumar(3, 4) # r = 7  
print(r * 2)    # 14 ✅
```

función()

– return →

valor útil

```
# Múltiples valores de retorno  
def estadisticas(nums):  
    return sum(nums), sum(nums)/len(nums), max(nums), min(nums) # retorna una tupla  
total, prom, mx, mn = estadisticas([85, 92, 78, 95]) # desempquetado
```


Alcance de variables – Scope

El scope determina desde dónde es accesible una variable en el programa.

SCOPE GLOBAL

```
empresa = 'TechBolivia' # global
```

SCOPE LOCAL – def mostrar_info()

```
empleados = 25    # local
print(empresa)    # ✅ global OK
print(empleados)  # ✅ local OK
```

```
mostrar_info()    ✅
print(empresa)    ✅
print(empleados)  ❌ NameError
```

La palabra **global** permite modificar una variable global desde dentro de una función:

```
contador = 0
def incrementar():
    global contador
    contador += 1
incrementar()
print(contador) # 1
```

Docstrings – documenta tus funciones

Describe qué hace la función, sus parámetros y lo que retorna. Parte esencial del código profesional.

```
def calcular_descuento(precio, porcentaje=10):  
    """  
    Calcula el precio final después de aplicar un descuento.  
    Args:  
        precio (float): Precio original en Bs.  
        porcentaje (float): % descuento. Def. 10%.  
    Returns:  
        float: Precio final con descuento.  
    Ejemplo:  
        >>> calcular_descuento(100, 20)  
        80.0  
    """  
    return precio * (1 - porcentaje/100)
```



`help(calcular_descuento)`

muestra el docstring completo



`print(calcular_descuento.__doc__)`

acceso directo al texto

Funciones incorporadas de Python

Python incluye funciones listas para usar sin importar nada — las más importantes:

print()

Muestra texto en pantalla

input()

Lee texto del usuario

int() float() str()

Conversión de tipos

len()

len('Python') → 6

range()

Secuencia numérica

type()

Tipo de dato

abs()

abs(-5) → 5

round()

round(3.567,2)→3.57

max() min()

Máximo y mínimo

sum()

sum([1,2,3]) → 6

sorted()

Lista ordenada

+ de 60 funciones incorporadas

💡 Usa `dir(__builtins__)` en el intérprete para ver TODAS las funciones incorporadas de Python.

Ejercicio integrador — Librería de facturación

🎯 Crear funciones separadas y componerlas en un sistema de facturación completo

```
IVA = 0.13 # constante global
def calcular_subtotal(precio, cantidad):
    return precio * cantidad
def aplicar_descuento(sub, pct=0):
    desc = sub * (pct/100)
    return sub - desc, desc
def calcular_iva(base):
    return base * IVA
def generar_factura(prod, precio, cant, desc=0):
    sub = calcular_subtotal(precio, cant)
    base, dsc = aplicar_descuento(sub, desc)
    iva = calcular_iva(base)
    total = base + iva
    print(f"TOTAL: Bs. {total:.2f}")
    return total
```

generar_factura()



calcular_subtotal()



aplicar_descuento()



calcular_iva()

💡 Composición de funciones: piezas pequeñas que hacen una sola cosa y se combinan para crear algo mayor.

✓ LO QUE APRENDISTE HOY

Módulo 5 en resumen



def / llamada

Definir y ejecutar funciones



Parámetros

Variables que recibe la función



return

Devolver resultados reutilizables



Defaults

Valores por defecto en parámetros



Scope

Local vs global



global

Modificar variables globales



Docstrings

Documentar funciones profesionalmente



Built-ins

len, sum, max, min, sorted, abs, round...



Próxima sesión: **Módulo 6 — Listas y Tuplas**